

计算机系统基础实 验报告

实验四：Cachelab



学生姓名：俞浩坤

学 号：22300240034

日 期：2023.12.5

一、实验结果：

在终端中执行./driver.py 后的截图：

```
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo python2 ./driver.py
Part A: Testing cache simulator
the following are original points, which will be transformed into the final points
Running ./test-csim
```

Points (s,E,b)	Your simulator			Reference simulator			
	Hits	Misses	Evicts	Hits	Misses	Evicts	
3 (1,1,1)	1	16	15	1	16	15	traces/yi2.trace
3 (4,2,4)	3	6	3	3	6	3	traces/yi.trace
3 (2,1,4)	0	5	4	0	5	4	traces/dave.trace
3 (2,1,3)	79	159	157	79	159	157	traces/trans.trace
3 (2,2,3)	128	110	106	128	110	106	traces/trans.trace
3 (2,4,3)	180	58	50	180	58	50	traces/trans.trace
3 (5,1,5)	149	89	84	149	89	84	traces/trans.trace
6 (5,1,5)	165997	120967	120962	165997	120967	120962	traces/long.trace
27							

```
part A final points: 27 * (40 / 27) = 40.0

Part B: Testing transpose function
Running ./test-trans -M 48 -N 48
Running ./test-trans -M 96 -N 96
Running ./test-trans -M 93 -N 99
Running honor-part

Cache Lab summary:
```

	Points	Max pts	Misses
Csim correctness	40.0	40	
Trans perf 48x48	14.0	14	434
Trans perf 96x96	14.0	14	1790
Trans perf 93x99	12.0	12	2586
Trans perf honor-part	5.0	5	3930
Total points	85.0	85	

二、部署实验环境：

根据助教给出的提示将 cachelab 挂载在 wsl 目录下执行，具体采样如下指令。

```
yuhaoshen@LAPTOP-QU516I60:~$ sudo cp -r /mnt/c/cachelab-handout /wsl
```

在 wsl 目录下执行命令时为了提升权限以使命令能够执行，需要在命令前加入 sudo 指令。

```
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo python2 ./driver.py
```

三、part A:

csim.h 部分：

```
struct block{
    int flag;
    int target;
    int count;
};
struct block **cache;

// final results
int hits = 0;
int misses = 0;
int evictions = 0;

// will be set in getopt() function
extern char *optarg;

// you can define functions here
void usage();
void parseline(int argc, char **argv);
void initcache();
void freecache();
void newcache(unsigned int);
void input();
```

在模板的基础上，新添加了结构体 block 作为高速缓存的块，其中 flag 代表有效位，target 代表标记位，count 对高速缓存块存在时间进行计数，然后用一个块的二维数组表示高速缓存，函数上添加了四个函数，其中 initcache 函数对高速缓存进行初始化，freecache 函数释放高速缓存空间，newcache 函数对高速缓存进行更新，input 函数每次读入操作。

csim.c 部分：

```
void initcache() {
    cache = (struct block**)malloc(sizeof(struct block) * numSet);
    for (int i = 0; i < numSet; i++)
    {
        cache[i] = (struct block*)malloc(sizeof(struct block) * associativity);
    }
    for (int i = 0; i < numSet; i++)
    {
        for (int j = 0; j < associativity; j++)
        {
            cache[i][j].flag = 0;
            cache[i][j].target = 0;
            cache[i][j].count = 0;
        }
    }
}
```

initcache 函数首先给高速缓存分配空间，之后对高速缓存的每个块进行初始化，由于初始为空，因而将有效位，标记位和存在时间都赋值为 0。

```
void freecache()
{
    for (int i = 0; i < numSet; i++)
    {
        free(cache[i]);
    }
    free(cache);
}
```

freecache 函数简单地将高速缓存申请的空间 free 掉即可。

```
void newcache(unsigned int a)
{
    int set = (a / blockSize) % numSet;
    int tar = a / (blockSize * numSet);
    for (int i = 0; i < associativity; i++)
    {
        if (cache[set][i].target == tar && cache[set][i].flag == 1)
        {
            cache[set][i].count = 1;
            if (verbose)
            {
                printf(" hit");
            }
            hits++;
            for (int j = 0; j < associativity; j++)
            {
                cache[set][j].count++;
            }
            return;
        }
    }
    if (verbose)
    {
        printf(" miss");
    }
    misses++;
}
```

newcache 函数首先根据传入的地址计算出组索引和目标值索引,接着判断 cache 中的相应位置的标记位是否等于目标值索引和其有效位是否为 1,如果都满足则命中, hits 数加 1 且组索引所对应组的所有块存在时间都加 1, 否则就发生了 miss, miss 数加 1。

```

    for (int i = 0; i < associativity; i++)
    {
        if (cache[set][i].flag == 0)
        {
            cache[set][i].flag = 1;
            cache[set][i].target = tar;
            cache[set][i].count = 1;
            return;
        }
    }
    int min = -1;
    int minposition = 0;
    for (int i = 0; i < associativity; i++)
    {
        if (cache[set][i].count > min)
        {
            min = cache[set][i].count;
            minposition = i;
        }
    }
    if (verbose)
    {
        printf(" eviction");
    }
    evictions++;
    cache[set][minposition].flag = 1;
    cache[set][minposition].target = tar;
    cache[set][minposition].count = 1;
    for (int i = 0; i < associativity; i++)
    {
        cache[set][i].count++;
    }
}

```

发生 miss 后如果原因是有效位为 0, 则采用有效位为 0 的块存储数据并进行更新, 如果原因是高速缓存满了的话, 则首先找到高速缓存中存在时间最长的块, 之后进行替换, eviction 数加 1, 再将替换的块更新, 最后将存在时间加 1, 其中如果显示标记为 1 则需要对情况信息进行打印。

```

void input()
{
    FILE* file = fopen(filePath, "r");
    char c;
    unsigned int address;
    int size;
    while (fscanf(file, " %c %x,%d", &c, &address, &size) > 0)
    {
        if (verbose)
        {
            printf("%c %x,%d", c, address, size);
        }
        if (c == 'I')
        {
            continue;
        }
        if (c == 'M')
        {
            newcache(address);
            newcache(address);
        }
        else
        {
            newcache(address);
        }
        printf("\n");
    }
    fclose(file);
}

```

input 函数读入数据, 采用文件指针的方式, 每次读入操作, 地址和大小, 如果显示标记为 1 则打印这些信息, 之后对操作进行判断, 如果是读指令则直接进入下个操作, 如果是修改数据则需要更新两次 cache, 其他都只需更新一次 cache。

```

int main(int argc, char *argv[])
{
    parseline(argc, argv);
    initcache();
    input();
    printSummary(hits, misses, evictions);
    freecache();
    return 0;
}

```

main 函数直接调用函数依次进行初始化，读入，打印结果，释放空间即可。

```

Part A: Testing cache simulator
the following are original points, which will be transformed into the final points
Running ./test-csim

```

Points (s,E,b)	Your simulator			Reference simulator			
	Hits	Misses	Evicts	Hits	Misses	Evicts	
3 (1,1,1)	1	16	15	1	16	15	traces/yi2.trace
3 (4,2,4)	3	6	3	3	6	3	traces/yi.trace
3 (2,1,4)	0	5	4	0	5	4	traces/dave.trace
3 (2,1,3)	79	159	157	79	159	157	traces/trans.trace
3 (2,2,3)	128	110	106	128	110	106	traces/trans.trace
3 (2,4,3)	180	58	50	180	58	50	traces/trans.trace
3 (5,1,5)	149	89	84	149	89	84	traces/trans.trace
6 (5,1,5)	165997	120967	120962	165997	120967	120962	traces/long.trace
27							

```

part A final points: 27 * (40 / 27) = 40.0

```

测试结果正常，part A 顺利完成。

四、part B

48*48 矩阵部分：

```

int num[12];
if(M == 48)
{
    for(int i = 0 ; i < 48 ; i = i + 12)
    {
        for(int j = 0 ; j < 48 ; j = j + 12)
        {
            for(int k = 0 ; k < 12 ; k++)
            {
                for(int l = 0 ; l < 12 ; l++)
                {
                    num[l] = A[i+k][j+l];
                }
                for(int l = 0 ; l < 12 ; l++)
                {
                    B[j+l][i+k] = num[l];
                }
            }
        }
    }
}

```

由于 cache 的 s 为 48，E 为 1，b 为 48，因而一个 cache 正好可以存储 12x12 矩阵大小的数据，为了减小 miss 数，对矩阵进行 12x12 的分块，同时将数据记录在数组中，避免组索引相同而发生的 miss。

代码完成后，对矩阵转置正确性和 miss 次数进行测试。

```

yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo ./tracegen -M 48 -N 48
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo ./test-trans -M 48 -N 48

Function 0 (1 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=48, E=1, b=48)
func 0 (Transpose submission): hits:4181, misses:434, evictions:386

Summary for official submission (func 0): correctness=1 misses=434

TEST_TRANS_RESULTS=1:434

```

miss 次数具体为 434 通过测试。

96*96 矩阵部分：

```

for(int i = 0 ; i < 96 ; i = i + 12)
{
    for(int j = 0 ; j < 96 ; j = j + 12)
    {
        for(int k = 0 ; k < 6 ; k++)
        {
            for(int l = 0 ; l < 12 ; l++)
            {
                num[l] = A[i+k][j+l];
            }
            for(int l = 0 ; l < 6 ; l++)
            {
                B[j+l][i+k] = num[l];
            }
            for(int l = 0 ; l < 6 ; l++)
            {
                B[j+l][i+k+6] = num[l+6];
            }
        }
    }
}

```

同样采用 12x12 的分块方式，但由于矩阵规模的限制在转换过程中需要将整个 12x12 的矩阵再细分为 6x6 的矩阵分别进行操作，首先对左上角的 6x6 矩阵直接进行转置即可，之后对右上角 6x6 矩阵先采取平移的方式，相当于用 B 矩阵的空间来进行数据的缓存。

```

for(int k = 0 ; k < 6 ; k++)
{
    for(int l = 0 ; l < 6 ; l++)
    {
        num[l] = A[i+6+l][j+k];
    }
    for(int l = 0 ; l < 6 ; l++)
    {
        num[l+6] = B[j+k][i+6+l];
    }
    for(int l = 0 ; l < 6 ; l++)
    {
        B[j+k][i+6+l] = num[l];
    }
    for(int l = 0 ; l < 6 ; l++)
    {
        B[j+k+6][i+l] = num[l+6];
    }
}

```

接着将左下角的 6x6 矩阵放入 B 的右下角，同时利用直接存储在 B 右上角的数据，将其放入 B 的左下角。

```

for(int k = 0 ; k < 6 ; k++)
{
    for(int l = 0 ; l < 6 ; l++)
    {
        num[l] = A[i+6+k][j+6+l];
    }
    for(int l = 0 ; l < 6 ; l++)
    {
        B[j+6+l][i+6+k] = num[l];
    }
}

```

最后直接将右下角的 6x6 矩阵进行转置即可。

代码完成后，对矩阵转置正确性和 miss 次数进行测试。

```

yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo ./tracegen -M 96 -N 96
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo ./test-trans -M 96 -N 96

```

```

Function 0 (1 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=48, E=1, b=48)
func 0 (Transpose submission): hits:21257, misses:1790, evictions:1742

Summary for official submission (func 0): correctness=1 misses=1790

TEST_TRANS_RESULTS=1:1790

```

miss 次数为 1790 通过测试。

93*99 矩阵部分：

```

if(M == 93)
{
    for(int i = 0 ; i < 99 ; i = i + 48)
    {
        for(int j = 0 ; j < 93 ; j = j + 12)
        {
            for(int k = 0 ; k < 48 ; k++)
            {
                for(int l = 0 ; l < 12 ; l++)
                {
                    if(j + l >= 93 || i + k >= 99)
                    {
                        break;
                    }
                    num[l] = A[i+k][j+l];
                }
                for(int l = 0 ; l < 12 ; l++)
                {
                    if(j + l >= 93 || i + k >= 99)
                    {
                        break;
                    }
                    B[j+l][i+k] = num[l];
                }
            }
        }
    }
}

```

对于不规则矩阵由于 miss 次数的上限 3000 较宽松，直接采取了较大的 48x12 的分块方式，在越出边界后直接跳出循环，发现能够通过。

对矩阵转置的正确性和 miss 次数测试结果如下：

```

yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo ./tracegen -M 93 -N 99
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout$ sudo ./test-trans -M 93 -N 99

Function 0 (1 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=48, E=1, b=48)
func 0 (Transpose submission): hits:15835, misses:2586, evictions:2538

Summary for official submission (func 0): correctness=1 misses=2586

TEST_TRANS_RESULTS=1:2586

```

miss 次数为 2586 通过测试，part B 顺利完成。

五、part C

```
for (int i = 0; i < 32; i++)
{
    for (int j = 0; j < 32; j++)
    {
        C[i][j] = 0;
    }
}
for (int i = 0; i < 32; i = i + 8)
{
    for (int j = 0; j < 32; j = j + 8)
    {
        for (int k = 0; k < 32; k = k + 8)
        {
            for (int l = 0; l < 8; l++)
            {
                for (int m = 0; m < 8; m++)
                {
                    num[m + 8] = 0;
                }
                for (int m = 0; m < 8; m++)
                {
                    for (int n = 0; n < 8; n++)
                    {
                        num[n] = A[l + i][n + j];
                        num[n] = num[n] * B[n + j][m + k];
                        num[m + 8] = num[m + 8] + num[n];
                    }
                }
                for (int m = 0; m < 8; m++)
                {
                    C[l + i][m + k] = C[l + i][m + k] + num[m + 8];
                }
            }
        }
    }
}
```

由于 cache 的 s 为 32，E 为 1，b 为 32，因而一个 cache 正好可以存储 8x8 矩阵大小的数据，为了减小 miss 数，对矩阵进行 8x8 的分块，在初始分块完成后发现 miss 数只降到了 10000 多，之后参考了书上 p441 页内容，发现循环次序的交换会大大影响 miss 数，于是将内层循环 m 和 n 所代表的行列进行交换，成功将 miss 数降低到 3930。

对矩阵乘法正确性和 miss 次数测试结果如下：

```
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout/honor-part$ sudo ./tracegen -M 32 -N 32
yuhaoshen@LAPTOP-QU516I60:/wsl/cachelab-handout/honor-part$ sudo ./test-mul -M 32 -N 32

Function 0 (1 total)
Step 1: Validating and generating memory traces
Step 2: Evaluating performance (s=32, E=1, b=32)
func 0 (multiply submission): hits:70829, misses:3930, evictions:3898

Summary for official submission (func 0): correctness=1 misses=3930

TEST_MUL_RESULTS=1:3930
```

miss 次数为 3930 通过测试，part C 顺利完成，至此完成本次实验所有内容。